CatanBot: A CNN-Based Agent for Playing Catan Using MCTS and Multitask Learning

Henry Jiang
Georgia Institute of Technology
Atlanta, Georgia, USA

Alexander Ge
Georgia Institute of Technology
Atlanta, Georgia, USA

Ray Dai
Georgia Institute of Technology
Atlanta, Georgia, USA

Jeffrey Chen
Georgia Institute of Technology
Atlanta, Georgia, USA

Tracy Yang
Georgia Institute of Technology
Atlanta, Georgia, USA

***Abstract*—CatanBot is a convolutional neural network model for the purpose of playing the game Catan using Monte Carlo Tree Search (MCTS), ResNet architecture, and multitask learning. By leveraging the feature-extraction capabilities of a CNN in tandem with supporting machine learning methods and heuristics, we seek to accurately interpret complex probabilistic board states and achieve win rates consistent with top human Catan players. The architecture also utilizes a dedicated neural network and MCTS system for handling trades, a hand tracking system, and heuristics for setup phase, building, trading, and robber placement.**

I. Introduction

Previous architectures consisting of CNN models finetuned on reinforcement learning with Monte-Carlo Tree Search (MCTS) for state evaluation have proved extremely effective by AlphaZero for Go and Chess [1]. These models utilized ResNet architectures for initial training before finetuning with reinforcement learning by having the model play games against itself. Later models such as AlphaGo [2] also suggest that reinforcement learning on its own is sufficient for training high performance models. We elected to utilize a similar ResNet architecture as AlphaZero for prediction of heuristic board state evaluations, which would allow us to perform MCTS to gauge the quality of specific moves.

For Catan in particular, typical deterministic strategies used in deterministic games like chess such as Minimax and alpha-beta pruning are not possible. This is due to the uncertainty of future outcomes for moves and the ambiguity of the best possible move as a result. However, strategies such as MCTS [3] and Markov Chains [4] have also shown promise in application to Catan play, with random walks helping to identify probabilistic winning moves across possible dice roll combinations. Due to the random aspect of Catan, presence of trades, and the complex board states and turn combinations, it is difficult to search further than one or two moves into the future for a model. Deep reinforcement learning strategies have also been applied to Catan to achieve high level performance, demonstrating the applicability yet relatively untapped potential of machine learning methods for playing Catan [5].

A. Dataset

Our original dataset consists of 43,947 anonymized 4-player Catan games scraped from Colonist.io, a web site for playing Catan against other human or AI players [6]. Unfortunately, the Github link for this dataset no longer appears to be active during the course of our research. Each game is encoded in a JSON file, consisting of the initial board state and every action taken by the players across the course of the game. The full hierarchy of data can be found in Fig. 1.

Fig. 1. Hierarchy of data present in a single JSON game entry in the original Colonist.io game dataset.

Dataset source: https://github.com/Catan-data/dataset/releases/tag/v1.0.0

II. Problem and Motivation

Catan offers a unique set of challenges compared to deterministic games such as chess and Go, where supervised learning has displayed success in mastering the game. Beyond having four players rather than two, Catan contains random chance through the randomized starting state of the board and dice rolls. These rolls determine the resources that a player receives each turn, potential loss of resources, and card stealing through robber movement. Furthermore, Catan has the important game mechanic of trades, which present a combinatorially

explosive set of circumstances and possible moves for an agent to consider in every turn. Effective trading considers modeling opponent needs, likelihood of trade acceptance, and how the trade repositions players towards victory.

Our project seeks to apply supervised learning and search based methods and heuristics to determine the effectiveness of machine learning in accurately modeling outcomes in non-deterministic game states and games with multi-player driven interactions. We are motivated by a personal interest in the game of Catan and the desire to produce bots capable of more sophisticated planning and moves than random move baselines and typical heuristic based bots. We would also like to qualitatively evaluate the performance ourselves by playing against the bot directly.

III. Methods

A. Data Preprocessing

Our neural network needed labeled data that utilized the final outcome of a game to evaluate specific board states and moves, so we processed the original dataset to create a new dataset of 8 million entries of specific turns in the dataset. In addition, we removed all games that did not satisfy the conditions of:

- No special game modes
- Must be completed (have a winning player)
- Exactly 4 players
- Minimum 20 turns
- No bots
- No parsing errors or corrupted states

This ensures that our data is uniform and of high quality, indicative of the conditions that we will be evaluating our model on.

To select specific turns from the dataset, we created a simulator that was able to simulate turns up until any specific turn and return the corresponding state vector (`CatanState`). We took our original dataset of raw games and converted them into `CatanState`s with a script. This state contains 1363 data points regarding tile types, edge states, corner states, bank cards, individual player information, etc.

We extracted around 8 million random turns out of 40 million total turns played in the original dataset, converting them into flat 1363 dimension vectors for a new dataset used in training and evaluation. In order to train our model with supervised learning, we derived a heuristic to establish the quality of a player's position subsequent to a move and associated it with every turn in our transformed dataset. The heuristic is discussed in more detail below.

B. Algorithms

1) Positional Evaluation Heuristic: We implemented a heuristic with tunable hyperparameters to weight the influence of specific factors to consider when evaluating the quality of a move or a player's current position. Catan is won by a player achieving 10 victory points (VP), of which can be achieved by building settlements/cities (each one VP), playing certain development cards (1 VP, hidden from other players), and holding the longest road and longest army (2 VP each, only one holder at a time). However, maintaining the lead in Catan generally exposes the player in the lead to "tall poppy syndrome", with other players collaborating against them; as such we weight the heuristic score based on the ultimate winner of the game and the current point leaderboard of a turn depending on how close the turn is towards the end of the game. If closer to the end of the game, the ultimate winner is weighted as more important in the scoring mechanism.

Players that did not ultimately win the game are also evaluated on a graduated scale; for example, a player that ended with 3/10 VP would have their evaluations penalized more severely than a player that ended with 8/10 VP. The safety of a lead in points is also similarly evaluated. For example, a player with 8 points while all others have 3 is graded as a much more favorable lead than a player with 8 points while all others have 7.

The full heuristic is defined as:

$$y = 0.50 \cdot s_{\text{outcome}} + 0.30 \cdot s_{\text{position}} + 0.20 \cdot s_{\text{economic}} \tag{1}$$

The blended confidence score is:

$$s_{\text{outcome}} = 0.5 \cdot \big(1 - \sigma(6 \cdot (t/T - 0.5))\big) + \text{true\_label} \cdot \sigma(6 \cdot (t/T - 0.5)) \tag{2}$$

The VP lead score is:

$$s_{\text{position}} = \text{clip}(0.4 \cdot \text{lead} + 0.4 \cdot \text{win\_proximity} - \text{penalty} + 0.4,\ 0,\ 1) \tag{3}$$

The economic state score is:

$$\begin{aligned} s_{\text{economic}} = \text{clip}(&0.35 \cdot s_{\text{prod}} + 0.20 \cdot s_{\text{diversity}} \\ &+ 0.10 \cdot s_{\text{port}} + 0.15 \cdot s_{\text{dev}} + 0.20 \cdot s_{\text{hand}},\ 0,\ 1) \end{aligned} \tag{4}$$

s_{outcome} blends the true winner label with a position-based estimate. This blend uses a sigmoid gate to shift towards the true label from the point leader as the game progresses. s_{position} rewards based on the VP lead and the closeness to the 10 VP goal. It also directly penalizes a shrinking of a held lead and rewards the closing of a gap towards the lead. s_{economic} encompasses the player's overall position quality and access to economic resources. Generally, a player with access to all resources has an easier time amassing victory points (s_{prod}). We also consider a player's access to ports (s_{port}), unplayed development cards (s_{dev}), and quality of current resources in hand (s_{hand}). We are able to determine the influence of the three factors discussed as hyperparameters: final winner (w), current points (p), and position strength (e) in the calculation of our heuristic. Our model was trained with initial parameters of $\{$w: 0.5, p: 0.3, e: 0.2$\}$ and included the weights inside each individual equation as hyperparameters as well. This heuristic was used to label each turn in our 8 million turn dataset for use in training our CNN.

2) Neural Network Architecture: Our neural network architecture maps our 1363 dimension input vector and outputs a scalar between $[0, 1]$ corresponding to our assigned heuristic evaluation. Our architecture starts by converting our vector into 256×256 dimension hidden space with two input projection

layers. The main portion consists of 12 blocks of pre-activation residual blocks: linear → dropout → ReLU → batch norm → linear → ReLU → batch norm. This follows the design of He et al. [7]. Skip connections allow our model to process with deeper architecture while maintaining context of the input vector.

We applied multitask learning as a machine learning method to our model by splitting the output of the ResNet to three parallel output heads, branching off to provide richer gradient signal during training. Each head corresponds to one of the factors in our position evaluation heuristic. The value head is our primary output, and is a 2-layer MLP that outputs our loss scalar from $[0, 1]$ via sigmoid activation. Our auxiliary head is a 2-layer MLP that predicts our outcome, position, and economic scores separately. Our win head directly correlates to the win/loss variable associated with the current player making the turn being evaluated. We trained with AdamW for 60 iterations, batch_size=2048, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and weight decay $= 10 \times 10^{-4}$. We later also used a cosine warmup schedule and clipped gradients with a reduced total iteration count (initial was 100) due to initial struggles with overfitting. Our train/test split was 80%/20%.

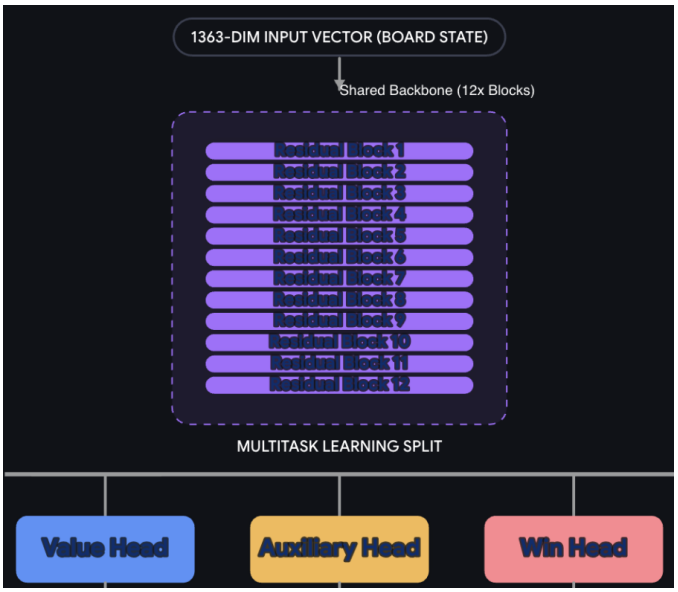

Fig. 2. Diagram of Neural Network Architecture.

At inference, the evaluator is wrapped in a `StateEvaluator` class for the CNN backend and using the base position evaluation heuristic for opponents' turn simulation. This allows fast opponent simulation within MCTS without calling the GPU and greatly accelerates evaluation time.

C. MCTS Agent Heuristics

CatanBot's MCTS agent uses an MCTS search function to select actions during the main game phase of a turn. MCTS runs for 100 iterations with a depth of 2 to predict the future state evaluations for various moves and dice roll combinations. At each leaf node, the neural evaluator scores resulting states and weights probability of victory. Opponent simulation utilizes MCTS with the untrained heuristic for a depth of 2.

1) Setup Phase Heuristics: During setup phase, the agent utilizes a dedicated placement heuristic as the setup lacks a natural tree structure for MCTS. Settlement placement scores are weighted by production rate (dice roll number quality), resource diversity, access to ports, and production of usable corners in vicinity (unclaimed by opponents). The first settlement is scored as a standalone, and the second is scored by complementary resources to the first settlement. Roads are weighted by the production value of the reachable corners in the direction of the road placed. Resource-specific ports are also weighted 2:1 proportional to the agent's production of that resource. 3:1 general ports receive a flat bonus rate.

2) Robber, Discard, Steal Heuristics: Non-main phase decisions such as robber placement, discarding excess cards, and who to steal from are also considered. Robber placement on a hex maximizes expected production and disruption to opponents (pip probability, building multiplier on tile). Discarding when a 7 is rolled iteratively discards the least desired resource for the player (desire score covered more in the next section). Stealing with the Knight or robber is done with a heuristic based on the opponent with the most resources and the opponent with the VP lead. Another simple heuristic was to play a Knight development card if available and the robber was on the player's production hex. The inclusion of heuristics discussed in this section and the previous propelled the model's win rate significantly and may have contributed to decreases in overfitting.

D. Trade CNN and MCTS

We trained a separate MCTS subsystem to handle player-to-player trade proposals and requests. For this, we utilized the following implementation:

- **Root decision node:** Choose trade from possible combination of materials.
- **Chance nodes:** Run MCTS based on accept/reject of trade to evaluate quality (trained acceptance model, uses evaluations of base CNN).
- **Leaf evaluation:** Score using CNN evaluation score; trades scored by proposal policy (softmax prior over candidate scores).

We use additional heuristics to penalize trades with the leader. We usually enumerate 2:1 and 1:1 resource swaps, and prune by top-K for the proposal policy. This system also runs for 100 iterations. The neural network architecture used for this model's acceptance criteria was identical to the one outlined above.

1) Desire Score and Surplus Tracking: The desire/surplus modules quantify per-resource need and excess for each player as a weighted sum across production methods and materials in hand. Excess is the amount of materials above the maximum necessary requirement for a build goal (2 wheat, 3 ore for a city, for example). These scores are used to weight the

acceptance rate of a trade and the discarding of the least desired resource.

2) Opponent Hand Tracking: Particle-filter based hand tracking maintains the possible hand assignments of opponents from the perspective of the current player. Each particle represents the full assignment of resources to all players and the tracker updates deterministically based on observable events like dice rolls, building purchases, bank trades, player trades, and discards. Uncertainty arises from robber steals where the tracker branches particles based on probability. Particles inconsistent with subsequent observations are pruned and particles cap at 200 in count. At each MCTS leaf evaluation, opponent hands are sampled from particle distribution to aid the trade MCTS system and the overall MCTS.

RESULTS

We evaluate the performance metrics of the model in evaluating games from the original dataset of *Colonist.io*, measuring predictive capability across the full course of the game. We also collect performance data of the full model against a random move baseline condition and against a greedy MCTS heuristic bot using the untuned heuristic. These experiments confirm meaningful learning by the CNN for predictive signals and that the MCTS is generally stronger than baselines.

4.1 CNN Full-Game Evaluation

We ran the CNN evaluator over every turn of 200 complete games (12,622 turns total), tracking whether the true winner was predicted accurately to win at each point in the game. All accuracy metrics substantially exceed the 25% top-1 / 50% top-2 random baseline. The updated metrics are summarized below in Table 3.

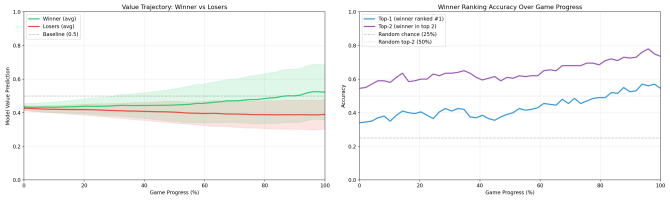

Fig. 3. **Figure 3:** (Left) Mean predicted position value for winning vs. losing players across game progress. Winners drift upward while losers drift downward as the game matures. (Right) Ranking accuracy (top-1 and top-2) of the winner prediction across game progress.

The average predicted value for eventual winners versus losers were at first assigned similar values (around 0.43 vs. 0.43), reflecting uncertainty early on. By the late game, winners converge to around 0.52 while losers settle near 0.39, a gap that widens steadily as the game progresses. It is notable that the scalars from range $[0, 1]$ did not directly converge to percentage chances of victory, but rather overall confidence scores. Figure 3 (right) shows that top-1 ranking accuracy grows from 34% to approximately 57% in the final turns, while top-2 accuracy grows from 54.5% to 78%. Both are well above the random baselines of 25% and 50% respectively and reflect strong evaluation capabilities.

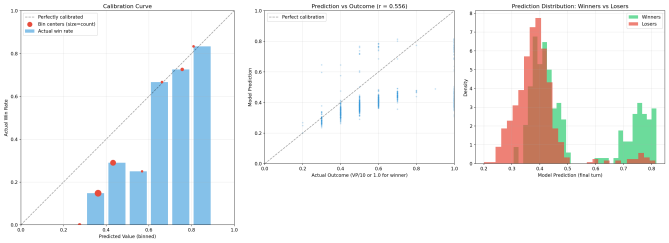

Fig. 4. **Figure 4:** (Left) Calibration curve comparing mean predicted win probability to actual win rate in each prediction bin. (Right) Scatter plot of final-turn predicted value vs. normalized heuristic outcome for all 200 games.

Figure 4 (left) is the calibration curve. Predictions above 0.65 are well-calibrated and players predicted at 0.75 confidence win at a 72.6% rate. Those predicted at 0.85 win at 83.3% as well, indicating the model underestimates actual win rates somewhat at high levels of confidence. Figure 4 (right) shows the scatter plot of final-turn predictions against the normalized heuristic outcome. Winners cluster at higher predicted values (0.6–0.82), while non-winners are spread across the lower ranges. The prediction-to-outcome correlation is 0.556 and the prediction-to-win correlation is 0.452.

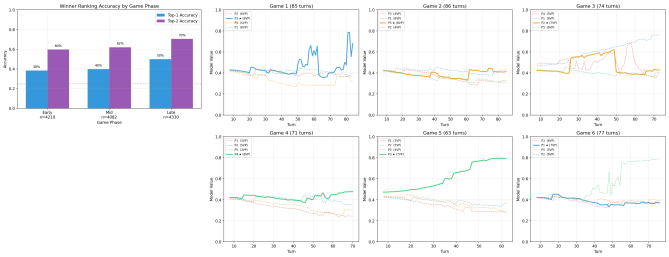

Fig. 5. **Figure 5:** (Left) Top-1 and top-2 accuracy broken out by game phase (Early, Mid, Late). (Right) Sample win-probability trajectories across individual games. The eventual winner is shown in blue.

Figure 5 (left) shows phase-based accuracy. Overall top-1 placement accuracy is 42.8% and top-2 placement accuracy is 64.1%. In the early game, top-1 is 38.3% (top-2: 59.7%); in the mid-game, top-1 rises to 39.8% (top-2: 62.0%); and in the late game, top-1 reaches 49.9% (top-2: 70.5%), with mean winner rank improving from 2.20 to 1.90 out of 4. Phase accuracy starts low and grows over time, which is expected behavior. Previous models exhibited strong overfitting and did not exhibit this trend of growing accuracy across phases. Figure 5 (right) shows evaluation scores across the course of several sample games. In well-decided games, the winner will generally have their score grow over time relative to the rest of the players. In closer games, predictions are noisier and often converge only in the final 10–20% of turns.

4.2 CNN MCTS Agent vs. Random Baseline

We evaluated the MCTS agent in 100 four-player games with one CatanBot and three bots executing a random move policy. This establishes a baseline to confirm that the agent is

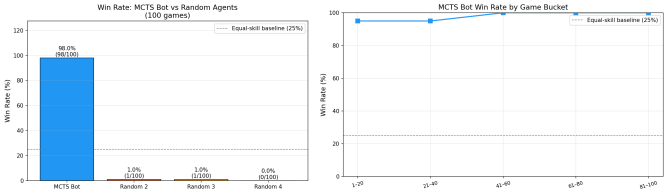

Fig. 6. **Figure 6:** (Left) Overall win rate of CatanBot vs. random agents. (Right) CatanBot win rate broken down by game-length bucket.

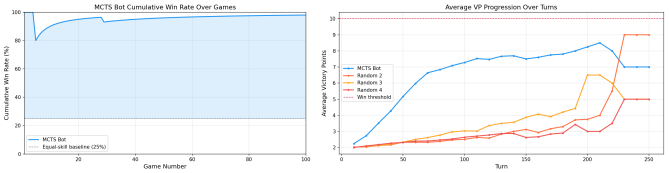

Fig. 7. **Figure 7:** (Left) Cumulative win rate over games played (vs. random). (Right) Average VP progression over game turns for CatanBot vs. random agents.

not simply making completely random moves or moves that are worse than a purely random baseline.

Figure 6 (left) shows CatanBot's overall win rate against random opponents. CatanBot achieves a win rate substantially above the 25% expected win rate with a 98.0% win rate, confirming the agent is playing purposefully rather than randomly.

Figure 7 (left) shows the cumulative win rate stabilizing quickly and Figure 7 (right) shows VP accumulation over turns. CatanBot consistently outpaces random agents in VP accumulation from the mid-game onward, reflecting effective resource conversion and build prioritization through the MCTS and positional heuristics.

Figure 8 (left) confirms that CatanBot consistently finishes with higher VP totals than random agents and Figure 8 (right) shows that games against random opponents tend to end faster, as the bot reaches 10 VP more efficiently without competitive pressure.

Figure 9 (left) shows that CatanBot collects resources at a higher rate than random agents. Figure 9 (right) shows interesting results in that CatanBot highly prioritizes development cards in its gameplan. This is indeed a known strategy at high levels, and confirms the MCTS trade and build selection is providing meaningful strategic direction.

4.3 CNN MCTS Agent vs. Heuristic MCTS

We also evaluated CatanBot against our base heuristic outlined previously using a greedy method of MCTS. We had CatanBot play 100 games against three of these bots to track win rate and other metrics. The heuristic baseline is a rule-based agent that always takes the highest immediate-value action based on a depth-2 MCTS search according to a fixed scoring function. These evaluations did not include any trades, as the heuristic bots had no encoded method of handling trades. This tests whether the CNN evaluator and MCTS lookahead add value beyond a strong hand-coded strategy. The agent was iteratively improved from an initial

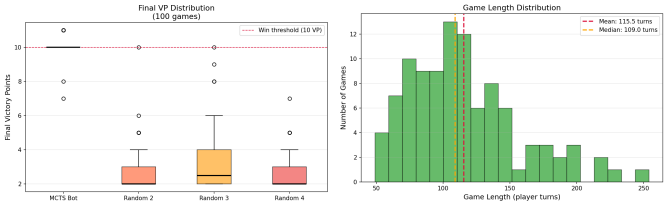

Fig. 8. **Figure 8:** (Left) Distribution of final VP counts for CatanBot vs. random agents. (Right) Distribution of game lengths (in turns).

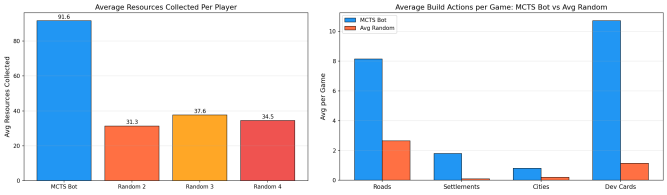

Fig. 9. **Figure 9:** (Left) Average cumulative resources collected over turns for each player type. (Right) Distribution of build actions taken.

severely overfitted state (approximately 19% win rate) through hyperparameter tuning and heuristic improvements.

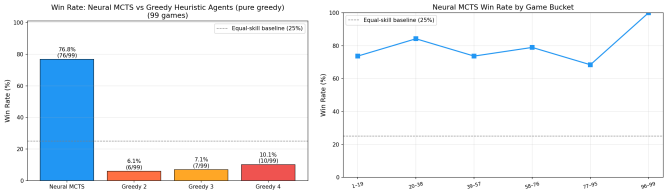

Fig. 10. **Figure 10:** (Left) Overall win rate of CatanBot vs. greedy heuristic agents. (Right) CatanBot win rate broken down by game-length bucket.

Figure 10 (left) shows CatanBot's win rate against greedy heuristic opponents. After retraining and heuristic tuning, the agent achieves approximately a 76% win rate in 4-player games against heuristic bots, far exceeding the 25% random baseline and substantially above the 35% win rate considered expert-level for human players. Figure 10 (right) shows that the win rate advantage is consistent across game lengths as well.

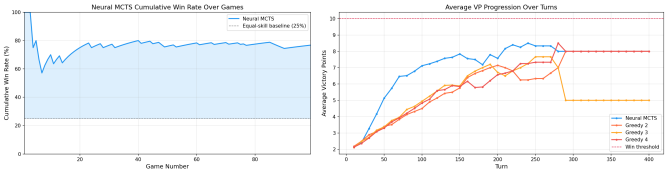

Fig. 11. **Figure 11:** (Left) Cumulative win rate over games played (vs. heuristic). (Right) Average VP progression over game turns for CatanBot vs. heuristic agents.

Figure 11 (left) shows the cumulative win rate over time. Figure 11 (right) shows that CatanBot and greedy heuristic bots accumulate VP at comparable rates early in the game. Both benefit from good setup placement, but CatanBot's

MCTS lookahead enables a decisive pull-ahead in the mid-to-late game.

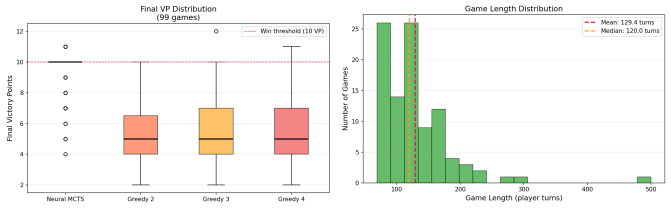

Fig. 12. **Figure 12:** (Left) Distribution of final VP counts for CatanBot vs. heuristic agents. (Right) Distribution of game lengths (in turns) against heuristic opponents.

Figure 12 (left) shows that against heuristic bots, CatanBot's VP distribution is more competitive than against random agents, as heuristic opponents also finish with relatively high VP counts. Figure 12 (right) shows that games against heuristic bots tend to run longer than against random agents, reflecting that heuristic bots play competently and contest resources more effectively.

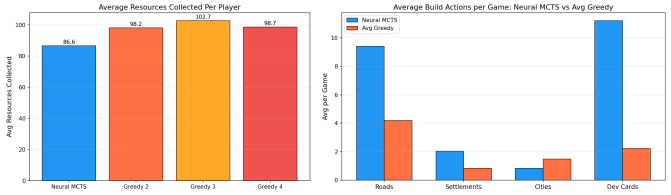

Fig. 13. **Figure 13:** (Left) Average cumulative resources collected for each player type vs. heuristic. (Right) Distribution of build actions taken vs. heuristic.

Figure 13 (left) shows that resource collection rates between CatanBot and heuristic bots are closely matched, as both use principled placement strategies. The key differentiator, shown in Figure 13 (right), is build action composition: CatanBot shows a more targeted build distribution, consistently prioritizing city upgrades and development cards, and also tends to build more roads. The results suggest stronger economic consistency for CatanBot and further reflect its favor towards development cards. This aligns with the MCTS evaluation favoring VP-efficient build paths.

Overall, these results confirm that CatanBot consistently outperforms non-machine-learning methods of automated Catan play. The CNN evaluator demonstrates above-baseline predictive accuracy for game outcomes, with accuracy improving as games progress. The full MCTS system achieves a 98% win rate against random opponents and approximately 76% against competent greedy heuristic bots. Qualitative evaluation of the agent's trade logs further confirms that the MCTS trade subsystem is making meaningful decisions, consistently exchanging surplus materials for needed ones at favorable ratios.

E. Discussion

Our results demonstrate that our CNN and MCTS system supported by heuristics is able to learn Catan and play at a competitive level against untrained heuristic MCTS systems. Our model was originally drastically overfitted with about a 19% win rate against the heuristic bots. After retrains from tuning hyperparameters and improving heuristics, we eventually reached 76% win rate against heuristics and a 98% win rate against completely random moves, suggesting meaningful improvement in performance from the application of the neural network. For reference, top Catan players are able to reach a 35% win rate at best, suggesting that our model is much more skilled than the base heuristic bot we implemented to start. From logs of proposed trades, the MCTS appears to be making informed decisions, tending to trade excess materials at a 1:1 or 2:1 ratio. It would be easier to assess this qualitatively with live play against the agent.

1) Next Steps: We were unable to successfully port our model fully onto Colonist.io and were only able to pull in data via a websocket and output the turn plan of the model in text form due to the hidden move making logic in Colonist. Although we did have a GUI for qualitative evaluations of specific moves made across a full game, we did not have the time to integrate the system fully with the websocket to allow the agent to visually indicate where it wanted to place specific settlements and roads. This would be a natural next step to implement in order to directly test our model against real players to document move choices clearer and record performance metrics against non-heuristic opponents. We could also use reinforcement learning to further tune our model weights with self-play with the final win-loss outcome as the reward/loss.

REFERENCES

[1] D. Silver et al., "Mastering chess and shogi by self-play with a general reinforcement learning algorithm," *arXiv preprint arXiv:1712.01815*, 2017.

[2] D. Silver et al., "Mastering the game of Go without human knowledge," *Nature*, vol. 550, pp. 354–359, 2017.

[3] I. Szita, G. Chaslot, and P. Spronck, "Monte-Carlo tree search in Settlers of Catan," in *Advances in Computer Games*, 2009.

[4] R. Nagel et al., "Markov chain analysis of Catan," 2021.

[5] H. Charlesworth, "Applying self-play reinforcement learning to the game Catan," *arXiv preprint arXiv:2204.03510*, 2022.

[6] MR MUCHO BUCHO, "Colonist.io Catan game dataset," 2025. [Online]. Available: https://github.com/Catan-data/dataset/releases/tag/v1.0.0

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. ECCV*, 2016.